

Binary Search Trees: A Formal Analysis of Search, Insertion, Deletion, Construction, and Validation Algorithms

Abstract

The Binary Search Tree (BST) is a foundational data structure that enforces an ordering invariant to enable efficient search, insertion, and deletion. This paper presents a rigorous formal analysis of seven fundamental BST operations: (1) BST property identification and the distinction between local and global invariant checking; (2) Search, which exploits the ordering invariant for binary-search-style element location; (3) Insertion at leaf positions while preserving the BST property; (4) Finding minimum and maximum elements via leftmost and rightmost traversal; (5) Deletion covering all three cases (leaf, one child, and two children with inorder successor replacement); (6) Construction of a height-balanced BST from a sorted array using divide-and-conquer; and (7) BST validation via both inorder traversal sorting checks and the superior min-max range propagation approach. For each operation, we provide complete Java implementations, formal correctness proofs by structural induction, detailed dry-run trace tables, TikZ-rendered tree diagrams, and comprehensive complexity analyses. All core operations achieve $O(H)$ time complexity where H is the tree height, motivating the study of self-balancing variants such as AVL and Red-Black trees.

Index Terms

binary trees, algorithms, data structures, computational complexity, tree traversal, recursive algorithms, formal analysis



1 Introduction

The Binary Search Tree (BST) is one of the most fundamental and widely-used data structures in computer science. By enforcing a simple ordering invariant — every node in the left subtree holds a value strictly less than the node, and every node in the right subtree holds a value strictly greater — BSTs enable efficient searching, insertion, and deletion operations that mirror the logic of binary search on sorted arrays, but with the flexibility of a linked, pointer-based structure.

BSTs find applications across a wide spectrum of computing:

- Database indexing: B-trees and B⁺-trees (generalizations of BSTs) form the backbone of database index structures, enabling logarithmic-time lookups on disk.
- Symbol tables: Compilers and interpreters use BSTs (or their balanced variants) to maintain symbol tables mapping identifiers to their types and memory locations.
- File systems: Directory structures often employ tree-based indexing for fast file lookup.
- Priority queues: Augmented BSTs support dynamic order-statistic queries and priority-based scheduling.
- Autocomplete systems: Trie-BST hybrids power real-time search suggestions.

This paper provides a rigorous formal analysis of seven fundamental BST operations:

- 1) BST Property and Validation (§3): Formal definition of the BST invariant and methods to verify whether a binary tree satisfies it.
- 2) Search (§4): Locating a target value by exploiting the ordering invariant.
- 3) Insertion (§5): Adding a new value while preserving the BST property.
- 4) Finding Minimum and Maximum (§6): Exploiting the structure to locate extremal elements.
- 5) Deletion (§7): Removing a node while maintaining the BST invariant, covering all three cases (leaf, one child, two children).
- 6) Constructing a Balanced BST from a Sorted Array (§8): Building an optimally balanced BST using divide-and-conquer.
- 7) BST Validation (§9): Two approaches — inorder traversal and min/max range checking — to verify the BST property.

For each operation, we provide complete Java implementations, formal correctness proofs, detailed dry-run analyses with TikZ-rendered tree diagrams, and rigorous complexity analyses.

2 Preliminaries

2.1 Binary Tree Node Structure

```
class TreeNode {
    int data;
    TreeNode left;
    TreeNode right;

    TreeNode(int x) {
        data = x;
        left = null;
        right = null;
    }
}
```

2.2 Formal BST Definition

Definition 1 (Binary Search Tree). A binary tree T is a Binary Search Tree if and only if for every node $v \in T$:

- 1) All nodes u in the left subtree of v satisfy $u.data < v.data$.
- 2) All nodes w in the right subtree of v satisfy $w.data > v.data$.
- 3) Both the left and right subtrees of v are themselves BSTs.

All node values are assumed to be distinct.

Definition 2 (Height of a BST). The height h of a BST with N nodes satisfies:

$$\lceil \log_2(N + 1) \rceil - 1 \leq h \leq N - 1$$

The lower bound is achieved by a perfectly balanced BST; the upper bound by a completely skewed (degenerate) BST.

3 BST Property and Visual Identification

3.1 Common Misconception

A frequent error is checking only the immediate children of each node: “is $v.left.data < v.data$ and $v.right.data > v.data$?” This is insufficient.

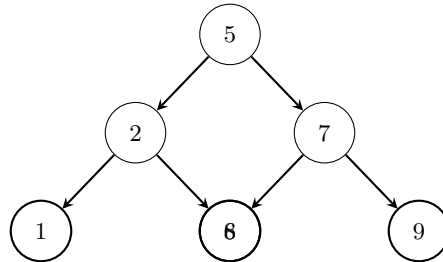


Fig. 1. NOT a BST despite each node satisfying the local child check. Node 8 is in the left subtree of 5, but $8 > 5$, violating the global BST property.

Lemma 3 (Global vs. Local Property). The BST property is a global constraint: every node in the left subtree must be less than the root, not just the immediate left child. Checking only immediate children leads to false positives.

3.2 Valid BST Example

4 Search in BST

4.1 Algorithm

The BST ordering invariant enables binary-search-style elimination: at each node, we either find the target, or eliminate one entire subtree:

```
boolean search(TreeNode root, int target) {
    if (root == null) return false;

    if (root.data == target) return true;
```

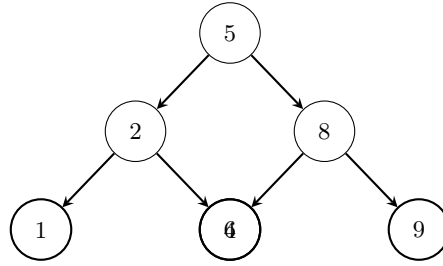


Fig. 2. A valid BST. Every node in the left subtree of 5 is < 5 , and every node in the right subtree is > 5 . This property holds recursively at every node.

```

if (target < root.data) {
    return search(root.left, target);
} else {
    return search(root.right, target);
}

```

Theorem 4 (Search Correctness). If v exists in BST T , then $\text{search}(T.\text{root}, v)$ returns true. If v does not exist, it returns false.

Proof. By structural induction on the tree height.

Base case: $\text{root} = \text{null}$. The target is not in the empty tree. Returns false. ✓

Inductive step: Assume correctness for trees of height $< h$. For a tree of height h :

- If $\text{target} = \text{root.data}$, return true. ✓
- If $\text{target} < \text{root.data}$: by the BST property, the target (if it exists) must be in the left subtree. We recurse on root.left , which has height $< h$. By the inductive hypothesis, the result is correct. ✓
- If $\text{target} > \text{root.data}$: symmetrically, we recurse on root.right . ✓

□

4.2 Example and Dry Run

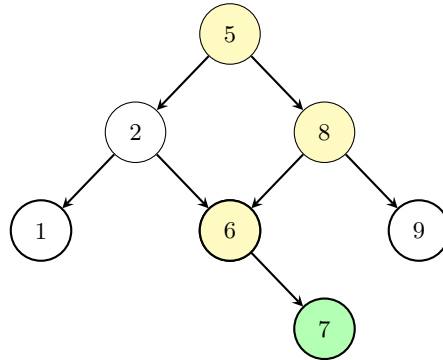


Fig. 3. Search path for $K = 7$ (highlighted in yellow, target in green).

Dry Run — $\text{search}(\text{root}, 7)$:

Step	Current Node	Comparison	Action
1	5	$7 > 5$	Go right
2	8	$7 < 8$	Go left
3	6	$7 > 6$	Go right
4	7	$7 = 7$	Return true

TABLE 1

Search trace for target $K = 7$. Visited 4 nodes out of 7 total.

4.3 Complexity Analysis

- Time: $O(H)$ where H is the height of the BST.
 - Best case (balanced BST): $O(\log N)$
 - Worst case (skewed BST): $O(N)$
- Space: $O(H)$ for the recursion stack (can be reduced to $O(1)$ with iterative approach).

5 Insertion in BST

5.1 Key Insight

New nodes are always inserted as leaf nodes. We traverse the tree using the BST property to find the correct position, then create a new node at the null position:

```
TreeNode insert(TreeNode root, int value) {
    if (root == null) {
        return new TreeNode(value);
    }

    if (value < root.data) {
        root.left = insert(root.left, value);
    } else if (value > root.data) {
        root.right = insert(root.right, value);
    }
    // If value == root.data, ignore (no duplicates)

    return root;
}
```

Theorem 5 (Insertion Correctness). After `insert(root, value)`, the resulting tree is a valid BST containing all original elements plus the new value.

Proof. By structural induction.

Base case: `root = null`. We create a single-node tree with the given value. A single node is trivially a valid BST. ✓

Inductive step: If `value < root.data`, we insert into the left subtree. By the inductive hypothesis, the left subtree remains a valid BST containing the new value. Since `value < root.data`, the BST property at the root is preserved. The right subtree is unchanged. Symmetric argument applies for `value > root.data`. ✓ □

5.2 Example and Dry Run

Insert value 6 into the following BST:

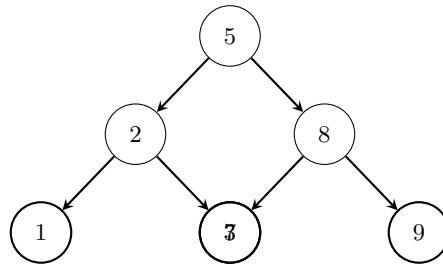


Fig. 4. BST before inserting 6.

Dry Run — `insert(root, 6)`:

Step	Node	Comparison	Action
1	5	$6 > 5$	Go right
2	8	$6 < 8$	Go left
3	7	$6 < 7$	Go left
4	null	—	Create node(6), return

TABLE 2

Node 6 is inserted as the left child of node 7.

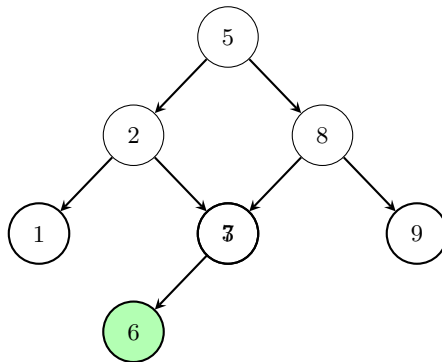


Fig. 5. BST after inserting 6 (highlighted in green).

5.3 Complexity Analysis

- Time: $O(H)$ — we traverse one path from root to a leaf.
- Space: $O(H)$ for recursion stack.

6 Finding Minimum and Maximum

6.1 Key Insight

By the BST property:

- The smallest element is the leftmost node (follow left pointers until null).
- The largest element is the rightmost node (follow right pointers until null).

6.2 Algorithms

```

TreeNode findMin(TreeNode root) {
    TreeNode temp = root;
    while (temp.left != null) {
        temp = temp.left;
    }
    return temp;
}
  
```

```

TreeNode findMax(TreeNode root) {
    TreeNode temp = root;
    while (temp.right != null) {
        temp = temp.right;
    }
    return temp;
}
  
```

Theorem 6 (Minimum Element). In a BST, the minimum element is located at the leftmost node.

Proof. Let v be the leftmost node (i.e., $v.\text{left} = \text{null}$). For any other node u in the tree:

- If u is an ancestor of v , then v is in the left subtree of u , so $v.\text{data} < u.\text{data}$.
- If u is in the right subtree of any ancestor of v , then $u.\text{data}$ is greater than that ancestor, which is greater than $v.\text{data}$.

Therefore $v.\text{data} \leq u.\text{data}$ for all nodes u , making v the minimum. ✓

□

6.3 Example

6.4 Complexity

- Time: $O(H)$ — we follow one branch to a leaf.
- Space: $O(1)$ — iterative, no recursion stack.

7 Deletion in BST

Deletion is the most complex BST operation because we must maintain the BST property after removing a node. There are three cases:

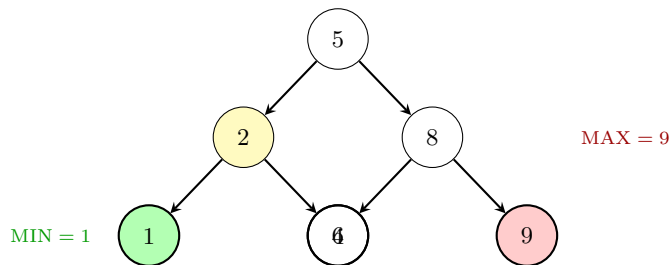


Fig. 6. Minimum (leftmost, green) and Maximum (rightmost, red) in a BST.

7.1 Case 1: Deleting a Leaf Node

The node has no children. Simply remove it by setting the parent's pointer to null.

7.2 Case 2: Deleting a Node with One Child

The node has exactly one child. Replace the node with its child (bypass the node).

7.3 Case 3: Deleting a Node with Two Children

This is the challenging case. We cannot simply remove the node. Instead:

- 1) Find the inorder successor (smallest node in the right subtree) or the inorder predecessor (largest node in the left subtree).
- 2) Copy the successor's value into the node to be deleted.
- 3) Recursively delete the successor from the right subtree (which reduces to Case 1 or 2).

Theorem 7 (Inorder Successor Property). The inorder successor of a node v in a BST is the smallest value greater than $v.data$. It is always the leftmost node in the right subtree of v .

7.4 Algorithm

```

TreeNode delete(TreeNode root, int value) {
    if (root == null) return null;

    if (value < root.data) {
        root.left = delete(root.left, value);
    } else if (value > root.data) {
        root.right = delete(root.right, value);
    } else {
        // Found the node to delete

        // Case 1: Leaf node
        if (root.left == null && root.right == null) {
            return null;
        }

        // Case 2: One child
        if (root.left == null) return root.right;
        if (root.right == null) return root.left;

        // Case 3: Two children
        // Find inorder successor (min of right subtree)
        TreeNode successor = findMin(root.right);
        root.data = successor.data;
        root.right = delete(root.right, successor.data);
    }

    return root;
}

```

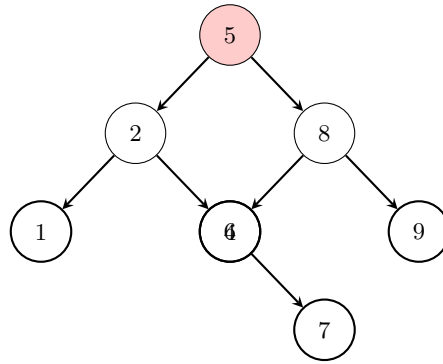


Fig. 7. BST before deleting root node 5. This is Case 3 (two children).

7.5 Example and Dry Run

Delete node 5 (root) from the following BST:

Dry Run:

- 1) Node 5 has two children $\Rightarrow$ Case 3.
- 2) Find inorder successor: smallest in right subtree of 5 = leftmost of subtree rooted at 8 = 6.
- 3) Copy 6 into node 5's position: root becomes 6.
- 4) Recursively delete 6 from the right subtree. Node 6 has one child (7) $\Rightarrow$ Case 2. Replace 6 with 7.

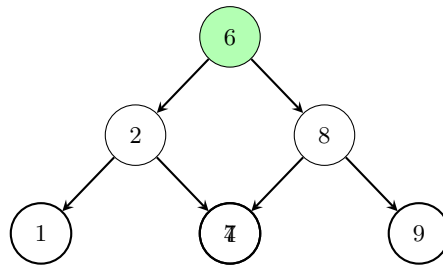


Fig. 8. BST after deleting 5. The inorder successor (6) replaced 5, and the original 6 was removed.

7.6 Correctness Proof

Theorem 8 (Deletion Correctness). After `delete(root, value)`, the resulting tree is a valid BST containing all original elements except the deleted value.

Proof. We prove each case:

Case 1 (Leaf): Removing a leaf does not affect any other node's BST relationships. $\checkmark$

Case 2 (One child): Bypassing a node with one child preserves the BST property because the child's entire subtree was already on the correct side of the deleted node's parent. $\checkmark$

Case 3 (Two children): The inorder successor s is the smallest value in the right subtree. Replacing the deleted node's value with $s.data$ preserves the BST property because:

- $s.data$ is greater than all values in the left subtree (since s was in the right subtree and the original node's value was also greater).
 - $s.data$ is less than all other values in the right subtree (since s is the minimum of the right subtree).
 - Recursively deleting s from the right subtree reduces to Case 1 or 2 (since s , being the leftmost node, has no left child).
- $\checkmark$

□

7.7 Complexity Analysis

- Time: $O(H)$ — at most two root-to-leaf traversals (one to find the node, one to find/delete the successor).
- Space: $O(H)$ for the recursion stack.

8 Constructing a Balanced BST from a Sorted Array

8.1 Problem Statement

Given a sorted array $A[0..N-1]$, construct a height-balanced BST containing all elements of A .

8.2 Key Insight

The middle element of the sorted array becomes the root. This ensures an approximately equal number of elements in the left and right subtrees, achieving minimum height:

```
TreeNode sortedArrayToBST(int[] A, int lo, int hi) {
    if (lo > hi) return null;

    int mid = lo + (hi - lo) / 2;
    TreeNode root = new TreeNode(A[mid]);

    root.left = sortedArrayToBST(A, lo, mid - 1);
    root.right = sortedArrayToBST(A, mid + 1, hi);

    return root;
}
```

Theorem 9 (Balanced Construction). The tree produced by `sortedArrayToBST` is a valid BST with height $\lceil \log_2(N+1) \rceil - 1$, which is the minimum possible height for N nodes.

Proof. **BST Property:** Since A is sorted, $A[\text{lo}..\text{mid}-1] < A[\text{mid}] < A[\text{mid}+1..\text{hi}]$. The left subtree contains only elements less than the root, and the right subtree contains only elements greater. By induction, each subtree is also a valid BST. ✓

Balance: At each level, the array is divided into two halves differing in size by at most 1. This produces a tree where every node's subtree heights differ by at most 1. ✓ □

8.3 Example

Input: $A = [1, 2, 3, 4, 5, 6, 7]$

Call	Range [lo, hi]	mid	Root Value
1	[0, 6]	3	4
2	[0, 2]	1	2
3	[0, 0]	0	1 (leaf)
4	[2, 2]	2	3 (leaf)
5	[4, 6]	5	6
6	[4, 4]	4	5 (leaf)
7	[6, 6]	6	7 (leaf)

TABLE 3

Recursive construction trace for $A = [1, 2, 3, 4, 5, 6, 7]$.

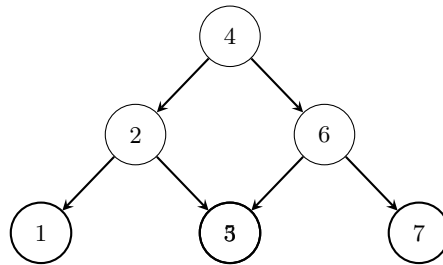


Fig. 9. Balanced BST constructed from sorted array $[1, 2, 3, 4, 5, 6, 7]$. Height $= 2 = \lceil \log_2 8 \rceil - 1$.

8.4 Complexity

- Time: $O(N)$ — each element is visited exactly once.
- Space: $O(\log N)$ — recursion depth equals tree height (balanced).

9 BST Validation

We present two approaches to verify whether a given binary tree satisfies the BST property.

9.1 Approach 1: Inorder Traversal Check

Lemma 10 (Inorder Property of BSTs). The inorder traversal of a valid BST produces elements in strictly increasing (sorted) order.

```
boolean isBSTInorder(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    inorder(root, result);

    for (int i = 1; i < result.size(); i++) {
        if (result.get(i) <= result.get(i - 1)) {
            return false;
        }
    }
    return true;
}

void inorder(TreeNode root, List<Integer> result) {
    if (root == null) return;
    inorder(root.left, result);
    result.add(root.data);
    inorder(root.right, result);
}
```

Complexity: $O(N)$ time, $O(N)$ space (for storing the traversal).

9.2 Approach 2: Min-Max Range Validation

A more elegant approach passes valid value ranges down the recursion:

```
boolean isBST(TreeNode root, int min, int max) {
    if (root == null) return true;

    if (root.data <= min || root.data >= max) {
        return false;
    }

    boolean leftValid = isBST(root.left, min, root.data);
    boolean rightValid = isBST(root.right, root.data, max);

    return leftValid && rightValid;
}
```

```
// Initial call:
// isBST(root, Integer.MIN_VALUE, Integer.MAX_VALUE)
```

Theorem 11 (Range Validation Correctness). $\text{isBST}(v, \min, \max)$ returns true if and only if all values in the subtree rooted at v lie in the open interval $(\min, \max)$ and the subtree satisfies the BST property.

Proof. By structural induction.

Base case: $v = \text{null}$. Returns true. An empty tree trivially satisfies any range constraint. ✓

Inductive step:

- If $v.\text{data} \leq \min$ or $v.\text{data} \geq \max$, the value is out of the valid range. Return false. ✓
- For the left subtree: all values must be in $(\min, v.\text{data})$. The recursive call $\text{isBST}(v.\text{left}, \min, v.\text{data})$ enforces this. ✓
- For the right subtree: all values must be in $(v.\text{data}, \max)$. The recursive call $\text{isBST}(v.\text{right}, v.\text{data}, \max)$ enforces this. ✓

□

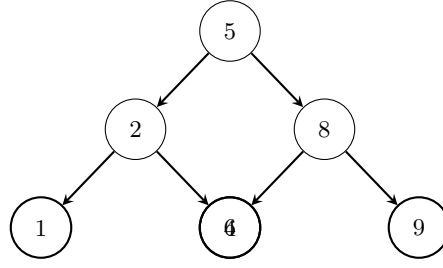


Fig. 10. Valid BST for min-max validation dry run.

Node	min	max	Valid?	Action
5	$-\infty$	$+\infty$	$-\infty < 5 < +\infty$ ✓	recurse
2	$-\infty$	5	$-\infty < 2 < 5$ ✓	recurse
1	$-\infty$	2	$-\infty < 1 < 2$ ✓	leaf, true
4	2	5	$2 < 4 < 5$ ✓	leaf, true
8	5	$+\infty$	$5 < 8 < +\infty$ ✓	recurse
6	5	8	$5 < 6 < 8$ ✓	leaf, true
9	8	$+\infty$	$8 < 9 < +\infty$ ✓	leaf, true

TABLE 4

Min-max validation trace. All nodes pass $\Rightarrow$ Valid BST.

9.3 Dry Run of Min-Max Approach

9.4 Complexity Comparison

10 Comprehensive Complexity Summary

10.1 BST vs. Balanced BST Variants

11 Conclusion

This paper presented a comprehensive formal analysis of seven fundamental Binary Search Tree operations: search, insertion, finding minimum/maximum, deletion (all three cases), balanced BST construction from sorted arrays, and BST validation via both inorder traversal and min-max range checking.

Key takeaways include:

- Search/Insert/Delete: All follow the same pattern of traversing one root-to-leaf path, achieving $O(H)$ time. The BST ordering invariant enables efficient binary-search-style elimination at each step.
- Deletion Case 3: The most subtle operation — replacing a two-child node with its inorder successor (or predecessor) elegantly reduces to simpler deletion cases while preserving the BST property.
- Sorted Array to BST: The divide-and-conquer approach of always choosing the middle element produces an optimally balanced tree in $O(N)$ time.
- BST Validation: The min-max range approach is superior to the inorder traversal method, requiring only $O(H)$ space versus $O(N)$, and operates in a single pass without materializing the full traversal.
- Height Sensitivity: All BST operations are $O(H)$, making tree balance critical. An unbalanced BST degenerates to $O(N)$ operations, motivating self-balancing variants like AVL trees and Red-Black trees.

Approach	Time	Space
Inorder traversal + sorted check	$O(N)$	$O(N)$
Min-Max range validation	$O(N)$	$O(H)$

TABLE 5

The min-max approach is superior in space complexity.

Operation	Average	Worst	Space
Search	$O(\log N)$	$O(N)$	$O(H)$
Insert	$O(\log N)$	$O(N)$	$O(H)$
Find Min/Max	$O(\log N)$	$O(N)$	$O(1)$
Delete	$O(\log N)$	$O(N)$	$O(H)$
Sorted Array to BST	$O(N)$	$O(N)$	$O(\log N)$
Validate (Inorder)	$O(N)$	$O(N)$	$O(N)$
Validate (Min-Max)	$O(N)$	$O(N)$	$O(H)$

TABLE 6

Complete BST operations complexity summary. H = tree height, ranging from $O(\log N)$ (balanced) to $O(N)$ (skewed).

Data Structure	Search	Insert	Delete
Unbalanced BST	$O(N)$ worst	$O(N)$ worst	$O(N)$ worst
AVL Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$
Red-Black Tree	$O(\log N)$	$O(\log N)$	$O(\log N)$

TABLE 7
Motivation for self-balancing BST variants.

References

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. MIT Press, 2022.
- [2] D. E. Knuth, The Art of Computer Programming, Volume 3: Sorting and Searching, 2nd ed. Addison-Wesley, 1998.
- [3] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Addison-Wesley, 2011.
- [4] S. S. Skiena, The Algorithm Design Manual, 3rd ed. Springer, 2020.
- [5] G. M. Adelson-Velsky and E. M. Landis, “An algorithm for the organization of information,” Soviet Mathematics Doklady, vol. 3, pp. 1259–1263, 1962.
- [6] L. J. Guibas and R. Sedgewick, “A dichromatic framework for balanced trees,” in Proc. 19th Annual IEEE FOCS, pp. 8–21, 1978.
- [7] T. N. Hibbard, “Some combinatorial properties of certain trees with applications to searching and sorting,” Journal of the ACM, vol. 9, no. 1, pp. 13–28, 1962.